Supplementary Material 5

The Open Software Stack Behind SWAFL Desktop

Languages and libraries and code and techniques and tools used to build the software and its documents

Naziru Halilu[1,2,*]

[1] Agricultural University of Athens, Athens, Greece

[2] Public University of Navarre, Pamplona, Spain

ORCID: 0009-0006-2324-9599

[*] Correspondence: halilunaziru73@gmail.com

Contents

S1 Purpose and scope . . . 3

S2 The software at a glance . . . 3

S3 Code map of the repository . . . 3

S4 Languages . . . 6

S5 Runtime stack . . . 7

S6 Libraries carried inside the program . . . 8

S7 Libraries of the whole repository, by language . . . 8

S7.1 Python . . . 8

S7.2 JavaScript and HTML . . . 10

S7.3 Programs called from the code . . . 11

S7.4 LaTeX packages and fonts . . . 11

S8 Installing everything: the exact commands . . . 11

S8.1 Python packages (pip) . . . 12

S8.2 JavaScript packages (npm) . . . 12

S8.3 Programs outside Python and JavaScript . . . 12

S8.4 Run the checks . . . 12

S9 Packages inside the bundled libraries and inside Daisy . . . 13

S9.1 GeoTIFF.js . . . 13
S9.2 SheetJS, Leaflet and Plotly.js . . . 14
S9.3 Libraries shipped with Daisy . . . 14
S10 Online services (optional) . . . 15
S11 Techniques used in the code . . . 15
S11.1 Structure of the program . . . 15
S11.2 Methods of the models . . . 16
S12 How the software is built . . . 17
S13 How the documents are built . . . 17
S14 Tools used to test and check . . . 18
S15 Licences and credit . . . 18
S16 Where to find it . . . 18

S1. Purpose and scope

This supplement stands alone. It belongs to the four articles of the series but is not tied to one of them: it records what SWAFL Desktop 1.1.0 is made of, in which languages it is written, which libraries and programs it carries, which techniques its parts use, and which tools were used to build, test and document it. Every figure below was counted on the source of version 1.1.0 or read from its `package.json`; where a number is approximate the text says so. Nothing in the stack is proprietary and nothing needs an account: all parts are open source or are the model and data that the series publishes.

S2. The software at a glance

SWAFL Desktop is one program with four layers, from the outside in.

1. **The desktop shell.** An Electron application (`main.js`, `preload.js`, `daisy-runner.js`, `serve.js`) that opens the window, saves and opens files, starts Daisy and opens the manual.
2. **The workbench.** About 90 JavaScript files in `app/js` (layers, properties, toolbox, dashboard, field on the map, sketch, the Daisy tabs, scientific figures, reports).
3. **The simulation engine.** A single page (`engine.html`, 1.5 MB, 7 225 lines) with the scripts of `app/engine` (crops, weather, the layers of the seasons, deterministic phenotyping). It runs in an inline frame and is joined to the workbench by a bridge.
4. **The Daisy model.** Daisy 7.1.14 for Windows with its library, packed unchanged beside the program and started as a separate process.

The same workbench and engine run in a web browser from a small local server (`npm run web`, or the `START_SWAFL_WEB` files) and online from the project page, so one code base serves the desktop, the browser and the web.

S3. Code map of the repository

Code is kept in four folders of the repository and nothing is hidden outside them. The counts are of the files tracked in the repository on the day of this supplement (the Daisy program folder and the vendored libraries are not counted).

Table S1: Where the code is, by folder.

Folder	Python files	JS files	What the code does and why it is there
`Software/v1.1.0/SWAFL_Desktop`	20	90	the program (shell and workbench and engine) and the tools that build its data and manual (14 100 lines of JavaScript and 2 500 of Python)
`Software/Old_Version_1.0`	16	12	the archived version 1.0 with its own build files (kept for reference)
`Manuscripts`	75	27	figures and tables and checks of the articles (`reproducibility`) and the graphical abstracts; 103 LaTeX files; scripts for the screenshots and checks in Node.js with Playwright
`Data`	4	0	the workbook of the data and the calculation sheets
`Results`	13	0	the 50 charts, the interactive charts, the videos, the 3D project and the Excel analysis (`08_Scripts`)

S4. Languages

Table S2: Languages and formats of the 1.1.0 source (the Daisy program and the manual PDF are not counted).

Language or format	Files	Lines (about)	Role and why it is used in the build
JavaScript (ECMAScript)	90	14 100	workbench and engine scripts and desktop shell; every equation of the layers and designs. Why: it runs in the browser and in Electron without a compiler, so one code base serves the desktop and the web.
HTML and inline script	2	7 200	the window (`index.html`) and the simulation engine (`engine.html`). Why: the engine is one page that can be opened alone and tested alone.
CSS	4	630	the classic grey and navy look and the dashboard. Why: one style sheet keeps the same classic look in every window.
Python 3.13	20	2 500	build tools of the data and of the manual (indexes and annotated screenshots and the equation table). Why: quick scripts for data, figures and file building that need no packaging.
JSON	4	20 500	libraries and indexes read by the program (for example the manual index). Why: readable data that JavaScript reads directly and that a person can edit.
Markdown	47	31 000	the chapters of the manual. Why: the text of the manual stays plain text under version control and builds to one PDF.
LaTeX	103	–	the four articles and their supplements (XeLaTeX with BibTeX). Why: exact equations and numbered references, and the same source gives the article and its supplement.
Batch and shell scripts	7	80	start and build files for Windows and macOS and Linux. Why: a double click starts the program without the user opening a terminal.
C++ (Daisy)	–	–	the source language of the Daisy model; used here only as the compiled program of Daisy 7.1.14. Why: it is the model whose runs the suite reads, so it is not rewritten.
SVG and CSV and XLSX	4 and 24 and 2	–	vector figures and plain tables and Excel workbooks. Why: open formats that any program can read.
Daisy log formats (`.dlf` and `.dwf`)	29	–	the daily logs and weather files that Daisy writes and reads. Why: the bridge reads them exactly as Daisy writes them.
Daisy set-up language (`.dai`)	library	–	the model description that Daisy reads; written by the workbench and by the user. Why: it is the only way to give Daisy a scenario, so the workbench writes it for the user.

The model of Daisy itself is written in C++ and is used here as a compiled program; SWAFL does not change its source. No framework (React or Vue or Angular) and no bundler is used: the pages are plain JavaScript loaded in order, so that the program can be read and changed with a text editor.

S5. Runtime stack

Table S3: What runs when SWAFL.exe is opened.

Part	Version	Role and why it is used in the build
Electron	31 (range `^31.0.0`)	window and Chromium and Node.js in one package; the portable build is made with it. Why: it packs a browser and Node.js into one program so that the web code runs as a desktop application without installation.
Chromium and Node.js	those of Electron 31	draw the canvases and run the JavaScript; Node gives file and process access to the shell only. Why: Chromium draws the canvases and Node.js lets the shell read and write files and start Daisy.
Daisy	7.1.14	soil and plant and atmosphere process model (University of Copenhagen); its Windows folder carries the program, a Python 3.13 runtime for its Python functions and the C++ libraries it needs. Why: the process model whose runs the decision layers show; it is used as built, not rewritten.

The shell and the page talk through a small bridge (`preload.js`, `contextBridge`) with about 20 named handlers (save a file, open a file, open the manual, find Daisy, run a Daisy job, read a log, list the runs, fetch an address, and a few window commands). The page itself has no direct access to the disk.

S6. Libraries carried inside the program

Table S4: Third party code that is copied into the program (all under open licences; the licence text sits beside each file).

Library	Version	Licence	Role and why it is used in the build
Leaflet	1.9.4	BSD 2-Clause	the map of the field (satellite and street layers; outlining the field). Why: a small map library that works offline with local code and gives the field outline tool.
GeoTIFF.js	bundled	MIT	reading the Sentinel-2 bands of the field from cloud-optimised GeoTIFF files in the browser. Why: satellite bands can be read in the page by range request, so no server of ours is needed.
SheetJS (xlsx)	0.18.5	Apache 2.0	importing weather from Excel and CSV and writing result tables. Why: weather and results travel in Excel files, which users already have.
Daisy 7.1.14 with Boost and SuiteSparse and the GCC runtime	as shipped	GPL and LGPL families (see the Daisy folder)	the process model and its numerical libraries; started as a separate program, not linked into SWAFL. Why: these are what Daisy itself needs to run, so they are shipped with it.

Everything else, including the figures, the report writer, the equations of the layers and the pipe hydraulics, is code written for this program.

S7. Libraries of the whole repository, by language

S7.1 Python

All third party Python packages imported anywhere in the repository (`Software`, `Manuscripts`, `Data`, `Results`) are listed. The versions of the first group are those recorded in `Manuscripts/reproducibility/r` (Python 3.13.16 on Linux x86-64); for the others no version is recorded in the repository and none is claimed. The standard library (os and sys and re and json and codecs and subprocess and others) is used throughout and is not listed.

Table S5: Python packages and what each is used for.

Package	Version	Role and why it is used in the build
numpy	2.5.3	arrays and the numerics of every analysis script. Why: fast arrays so that the analysis of 10 190 runs runs in seconds.
scipy	1.18.1	statistics (Spearman correlation and tests) and curve fitting and root finding (`brentq`, `curve_fit`, `minimize`) and `solve_ivp` and sparse matrices (the checks of the closed forms). Why: tested solvers and fitting routines instead of hand-written ones, which makes the checks independent.
pandas	3.0.5	reading the ensemble of 10 190 runs and summaries. Why: tables of runs are grouped and joined by name, which removes indexing mistakes.
scikit-learn	1.9.1	gradient boosting and random forest regression with grouped cross validation (`GroupKFold`), partial dependence and PCA on the ensemble. Why: standard learners and cross validation give results that others can repeat.
statsmodels	0.15.0	ordinary least squares models and their confidence intervals. Why: least squares with full statistical output for the effects reported in the articles.
matplotlib	3.11.2	every figure; `mpl_toolkits.mplot3d` for the 3D project; `animation` with the FFMpegWriter for the videos. Why: one library makes the print figures and the videos with a shared style.
Pillow (PIL)	12.3.0	screenshots and figure assembly and annotation. Why: puts screenshots and panels together without a graphics program.
openpyxl	not pinned	writing the Excel workbooks of the data and analysis. Why: writes the Excel files of the data with formulas and formats.
plotly	not pinned	the interactive charts and the 3D project saved as web pages. Why: charts that can be zoomed and rotated in a web page without a server.
pypdf	not pinned	preparing A4 versions of the articles. Why: sets the page size and joins pages of the PDFs.
wordfreq	not pinned	word frequencies (`zipf_frequency`) for a check of the wording in the analysis folder. Why: a check that the wording is plain and common.
playwright (Python)	not pinned	screenshots of the program for the manual. Why: takes the screenshots of the real program, so the manual shows what the user sees.

Small modules written for the repository (`common.py`, `load.py`, `figstyle.py`, `p_style.py`, `pm_style.py`, `annot.py`, `spec_atlas.py`, `textjs.py` and the like) hold the shared figure style and the loaders, so that every figure of the series has the same look.

S7.2 JavaScript and HTML

Table S6: JavaScript and HTML libraries and runtimes, end to end.

Library or runtime	Version	Role and why it is used in the build
Electron	31	shell of the desktop program (`require('electron')`: app and window and ipcMain and contextBridge and dialogs). Why: it packs a browser and Node.js into one program so that the web code runs as a desktop application without installation.
electron-builder	24.13.3	development only: packs `SWAFL.exe` (portable) and the installer. Why: the standard way to make a Windows portable file and installer from the same folder.
Node.js built-in modules	of Node 22 and Electron 31	`fs` and `path` and `child_process` and `os` and `zlib` and `http` and `vm` and `url` and `crypto`: the local server, the Daisy runner, the build and test scripts. Why: no extra package is needed for files and a local server, so the program has no hidden dependencies.
Leaflet	1.9.4	the map of the field in the program. Why: a small map library that works with local code and gives the field outline tool.
GeoTIFF.js	bundled	Sentinel-2 bands read in the browser. Why: satellite bands can be read in the page by range request, so no server of ours is needed.
SheetJS (xlsx)	0.18.5	Excel and CSV import and export. Why: weather and results travel in Excel files, which users already have.
Playwright for Node	not pinned	test and screenshot scripts that drive the real program in headless Chromium. Why: drives the real program as a user would, so a page error is caught before release.
Plotly.js	4.1.1	the interactive charts and the 3D project in `Results` (the library file is stored beside the pages so that they work offline). Why: the engine of the interactive charts, stored locally so they work offline.

Besides these, the program uses only what the browser provides: the 2D canvas, `Blob` and download links, `DecompressionStream` for compressed data, `localStorage` for small settings, `requestAnimationFrame` for the playing of a season, and `fetch` for the optional online services. There is no CDN call at run time: the libraries above are stored in the program folders.

S7.3 Programs called from the code

Table S7: Command-line programs that scripts call.

Program	Role and why it is used in the build
Daisy 7.1.14 (`daisy-bin.exe`)	started by `daisy-runner.js`; reads a set-up file and writes the logs. Why: it is the model itself, run as a separate process so a failed run cannot stop the program.
ffmpeg (libx264)	through matplotlib in `videos.py`: the videos of `Results/05_Videos`. Why: matplotlib needs it to write the MP4 files.
pandoc	the manual: Markdown chapters to one PDF with the XeLaTeX engine. Why: turns the Markdown chapters into the PDF of the manual in one step.
XeLaTeX and BibTeX (TeX Live)	the manual and the articles and the supplements. Why: XeLaTeX uses system fonts and Unicode, and BibTeX keeps the references consistent.
Tesseract and poppler (`pdftoppm`)	checking and annotating figures in `annotate_figs.py`. Why: reads text in figures and turns PDF pages into pictures for the checks.
Chromium (through Playwright)	screenshots and the tests of the program. Why: the same engine as the program, so the tests see the same pictures as the user.

S7.4 LaTeX packages and fonts

The articles and supplements load these packages (from `pre.tex` and `supp_style.tex`): amsmath and amssymb and array and bm and booktabs and caption and cleveref and enumitem and fancyhdr and float and fontspec and geometry and graphicx and hyperref and lineno and longtable and mathtools and microtype and multirow and natbib and pdflscape and setspace and siunitx and subcaption and tabularx and tcolorbox and tikz and titlesec and xcolor and xurl. The fonts are TeX Gyre Termes for the text and TeX Gyre Heros for sans serif and Latin Modern Mono for code. The manual uses DejaVu Serif and DejaVu Sans Mono.

S8. Installing everything: the exact commands

These commands install every package of the tables above under the names that the package indexes use (note that scikit-learn is imported as `sklearn` and Pillow as `PIL`, and PyMuPDF as `pymupdf`). Python 3.13 and Node.js 22 are assumed. A user of the finished program needs none of this: `SWAFL.exe` carries what it needs.

S8.1 Python packages (pip)

```
python3 -m pip install -r Manuscripts/reproducibility/requirements_all.txt
python3 -m playwright install chromium
```

which is the same as

```
python3 -m pip install numpy==2.5.3 scipy==1.18.1 pandas==3.0.5 \
    scikit-learn==1.9.1 statsmodels==0.15.0 matplotlib==3.11.2 pillow==12.3.0
python3 -m pip install openpyxl plotly pypdf wordfreq playwright pymupdf
```

On some Linux systems pip needs the flag `--break-system-packages` or a virtual environment (`python3 -m venv .venv` and then `source .venv/bin/activate`).

S8.2 JavaScript packages (npm)

```
cd Software/v1.1.0/SWAFL_Desktop
npm install                      # electron ^31.0.0 and electron-builder ^24.13.3
npm run get-app                  # builds SWAFL.exe
npm install -g playwright        # for the test and screenshot scripts
npx playwright install chromium
```

Leaflet, GeoTIFF.js and SheetJS are stored in the program folders and need no installation.

S8.3 Programs outside Python and JavaScript

```
sudo apt-get install ffmpeg pandoc texlive-xetex texlive-latex-extra \
    texlive-fonts-extra texlive-bibtex-extra tesseract-ocr poppler-utils \
    fonts-dejavu fonts-texgyre
```

On Windows the same programs come from their own installers (ffmpeg and pandoc and MiKTeX or TeX Live and Tesseract and poppler); Daisy 7.1.14 is already inside the program folder.

S8.4 Run the checks

```
cd Manuscripts/reproducibility
bash run_all.sh ./work           # regenerates the numbers and figures
```

S9. Packages inside the bundled libraries and inside Daisy

A library that is copied into the program carries packages of its own. They are listed here so that the stack is complete from the program down to its smallest part. The libraries are minified single files, so the packages were identified from their code and their licence notices (not from a package list, which a minified file does not have); no version is claimed for them.

S9.1 GeoTIFF.js

GeoTIFF.js (MIT licence, EOX IT Services) reads Sentinel-2 bands from remote GeoTIFF files by HTTP range requests. The program calls `GeoTIFF.fromUrl` (the maps and the monitoring of the field and the site chain) and `GeoTIFF.fromArrayBuffer`. Inside the file are these packages.

Table S8: Packages found inside `app/vendor/geotiff.js`.

Package	Evidence in the file	Role and why it is used in the build
pako (zlib in JavaScript)	Inflate and Z_SYNC_FLUSH code	decompresses DEFLATE tiles, a common compression of cloud-optimised files. Why: a tile cannot be read without it.
LERC (Esri, Apache 2.0)	the LERC decoder with its Esri notice	decodes the LERC compression of some rasters. Why: the same reader then works for more sources.
Zstandard decoder (zstddec)	`ZSTDDecoder`	decodes Zstandard tiles. Why: newer cloud-optimised files use it.
Half-float support (a float16 package)	`getFloat16`	reads 16-bit floating point samples. Why: some rasters store them.
Header parser (parse-headers style)	splitting of the header text at line ends	reads the `Content-Range` header of a range request. Why: the reader must know how long the whole file is.
Block cache (quick-lru style)	a maximum-size cache object	keeps recently read blocks. Why: fewer repeated requests for the same bytes.
XML helper (xml-utils style)	`findTagsByName`	reads the GDAL metadata of a file (scale and offset and no-data value). Why: the reflectance has to be scaled correctly.
Browser interfaces	`fetch` and `createImageBitmap`	range requests and the decoding of JPEG tiles by the browser. Why: no JPEG decoder has to be shipped.

A package that GeoTIFF.js can use for worker threads (a web-worker shim) was searched for and not found, so the program reads the tiles in the main thread.

S9.2 SheetJS, Leaflet and Plotly.js

- **SheetJS** (Apache 2.0) carries a table of character code pages (`cptable` 1.15.0, `DO_NOT_EXPORT_CODEPAGE` set), which lets it read old Excel and CSV files in other encodings. Why: weather files from other countries do not use one encoding.
- **Leaflet** 1.9.4 has no packages of its own; it is one file with its style sheet and images.
- **Plotly.js** 4.1.1 (Results folder only) bundles its own graphics and mapping code, including the MapLibre GL code that its map charts use. It is not part of the program.

S9.3 Libraries shipped with Daisy

The Daisy folder of the program (`daisy/bin`) carries the program and the libraries it was built with. They are listed from the file names of the folder.

Table S9: Files of the Daisy folder and what they are.

File or package	Role and why it is used in the build
`daisy-bin.exe`	the Daisy program, started by the workbench. Why: the process model itself.
`libcore.dll` and `libwindows.dll`	the core and Windows parts of Daisy (judged from the file names). Why: Daisy was built as a program and libraries.
`libboost_filesystem-mt.dll` (Boost)	file and folder handling. Why: Daisy reads and writes many files.
`libcxsparse.dll` and `libsuitesparseconfig.dll` (SuiteSparse)	sparse matrix solving. Why: the Richards and solute equations are solved implicitly with sparse matrices.
`libgomp-1.dll` (GNU OpenMP)	parallel loops. Why: speeds up the solvers on several cores.
`libstdc++-6.dll` and `libgcc_s_seh-1.dll` and `libwinpthread-1.dll` (GNU compiler runtime)	the C++ runtime. Why: the program was built with the GNU compilers for Windows.
`vcruntime140.dll` and `vcruntime140_1.dll` (Microsoft)	the Visual C++ runtime. Why: needed by the Python runtime that Daisy carries.
`python313.dll` and `python.exe` and the `Lib` and `DLLs` folders (Python 3.13)	the Python runtime and its standard library. Why: Daisy can call Python functions inside a set-up.

S10. Online services (optional)

Table S10: Services the program can call. All are optional; without a connection every other number is computed on the computer.

Service	Role and why it is used
Open-Meteo (forecast and archive and geocoding and elevation)	weather of any place (measured archive and forecast); place names; elevation. Why: free weather with no key, so the program needs no account.
Esri World Imagery and OpenStreetMap tiles	the satellite picture and the street map under the field. Why: open background maps so the user can see the field.
Microsoft Planetary Computer and Element 84 Earth Search (STAC)	Sentinel-2 L2A scenes for the vegetation and water indices. Why: open access to Sentinel-2 scenes in a standard catalogue (STAC).
SoilGrids (ISRIC)	soil of the field and its retention curves. Why: a soil estimate for any field in the world when the user has no soil data.
ESA WorldCover and Overpass (OpenStreetMap)	land cover and roads and water sources near the field. Why: tells the user what surrounds the field (roads and water) without a survey.
Nominatim and Photon	finding a place by name. Why: lets the user type a place name instead of coordinates.

S11. Techniques used in the code

S11.1 Structure of the program

- **Two halves joined by a bridge.** The workbench (shell page) and the engine (inline frame) share state through one object that the bridge sends to the engine and reads back; a setting changed in Properties is applied to every layer, the dashboard, the report and the designs, so they cannot disagree.
- **Canvas drawing.** Every picture (the field and the layers and the designs and the scientific figures) is drawn on a 2D canvas by code, so a figure can be exported as a picture or as numbers from the same data.
- **Lazy loading and guards.** Heavy scripts load when a tab is first opened; a guard module stops a failed script from freezing the window; long runs are cut into steps so the window stays responsive.
- **Provenance by construction.** Every number that is shown carries a label that leads to a numbered equation of the manual; the table of equations is generated from one file (`eqs.json`).

- **The rule of the Daisy layers.** A Daisy layer shows a result only from a Daisy run: results are read back from the log files of the run, never typed in.
- **Seeded randomness.** Projected seasons reuse the simulated seasons in a fixed random order from a seeded generator, so a projection is exactly repeatable.

S11.2 Methods of the models

- Soil hydraulics from texture with the Saxton and Rawls pedotransfer functions; the van Genuchten and Mualem retention and conductivity; the Richards equation solved in the soil column tab of the workbench.
- Surface irrigation: Kostiakov and Green and Ampt intake, Manning flow in the furrow, advance and recession by volume balance (Walker and Skogerboe).
- Pressurised irrigation: Hazen and Williams friction (C of 140 and 150), speed limits, pump head with lift and friction and working head, emitter and sprinkler layouts.
- Crop water: the FAO paper 56 daily balance with the crop coefficient method and the yield response of FAO paper 33; the FAO Penman and Monteith evapotranspiration.
- Satellite chain: scene search by STAC, band reading by range requests, cloud mask from the scene classification, vegetation and water indices per pixel, zonal statistics inside the outlined field.
- Experiment planning for Daisy: a catalogue of the factors Daisy can vary, a check of every design against it, an exact run budget, and a summary of the effect of every input from the logs.
- Verification by independent references: worked examples and held-out simulations and stored Daisy output, with the defect found in the denitrification function reported in the articles.

S12. How the software is built

Table S11: The build of the program.

Step	What happens and why
Start from source	`START_SWAFL.bat` (or `.command`) starts the program from the folder; it finds Node.js or fetches a portable copy of about 30 to 40 MB into `tools/node`. Why: a user can run the program before any build.
Install the build tools	`npm install` fetches Electron and electron-builder (24.13.3) listed in `package.json`. Why: the build tools stay out of the program and are needed only to make the file.
Build	`npm run get-app` runs electron-builder for Windows and writes `SWAFL.exe`. Why: one command makes the same file on every machine.
Targets	`portable` (one file and no installation) and `nsis` (an installer); a macOS `dmg` target is also defined. Why: a portable file for no-install use and an installer for those who want a shortcut.
Packed into the file	the files of the shell and `app/**/*` and the icon; as extra resources the manual PDF and the `daisy` folder. Why: the program must work offline with Daisy and the manual inside.
Web build	none needed: `npm run web` serves the same `app` folder with `serve.js`. Why: the same files serve the browser, so the web and desktop versions cannot drift apart.

The program is not code-signed, so Windows may show a SmartScreen notice, which is cleared with More info and then Run anyway.

S13. How the documents are built

- **The manual.** The chapters are Markdown files. A Python tool (`build_single.py`) joins them, adds the icons of the layers and the equation table, and `pandoc` 3.1.3 with the XeLaTeX engine (TeX Live 2023) makes one PDF with DejaVu Serif and DejaVu Sans Mono. Other tools write the index of sections that the program uses to open the manual at the page in use, and a short script with PyMuPDF (run at build time and not stored in the repository) reduces the size of the pictures so that the PDF opens from GitHub on any device.
- **The screenshots of the manual.** Taken from the running program with Playwright and headless Chromium, then annotated by Python scripts (`annot.py`, `new_capture.py`).

- **The articles and supplements.** LaTeX with XeLaTeX and BibTeX (`build.sh`); the tables and the numbers are written into the source from the data files by scripts, so that the text, the tables and the figures use the same numbers.
- **The figures of the series.** Python in the scripts of the repository, with the packages of the Python table above.

S14. Tools used to test and check

- Playwright with headless Chromium drives the real program: every layer is opened and run, the dashboard and the figures are read back, and page errors in the console are counted (the target is none).
- Node.js scripts run the equations of the designs outside the page and compare them with worked examples.
- Python scripts compare results with the stored Daisy output (`compare_results.py`, `audit_numbers.py`) and check the data against a table of checksums (`CHECKSUMS.sha256`).
- A local static server (`python -m http.server`) serves the program for these tests so that the same files are tested as are shipped.

S15. Licences and credit

The code of SWAFL and the articles are by Naziru Halilu and are released under the MIT licence of the repository. Leaflet and GeoTIFF.js and SheetJS keep their own licences and notices in the folders `app/vendor` and `app/engine/vendor`. Daisy keeps its licence in its own folder and is credited to the Daisy project of the University of Copenhagen (Hansen et al., 2012). The online services keep the terms of their owners and are credited where their data appear.

S16. Where to find it

All parts are in the repository *Coupled Water Fluxes, Pore-Scale Dynamics, Precision Irrigation, Mulching Feedback Loops*: the program in `Software/v1.1.0/SWAFL_Desktop`, the manual in `Software/v1.1.0/Manual`, the articles and supplements in `Manuscripts/pdf`, and the data in `Data` and `Results`. The way to rebuild the program is given in the install chapter of the manual.

References

Søren Hansen, Per Abrahamsen, Christian T. Petersen, and Merete Styczen. Daisy: model use, calibration, and validation. *Transactions of the ASABE*, 55:1317–1335, 2012. doi: 10.13031/2013.42244.